

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Lab Task

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ikram Ul Haq

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

Implementing Ansible on AWS EC2: Installing Docker and VS Code on the Central Node and Installing Them on Managed Nodes M1, M2 and M3 Using an Ansible Playbook

1. Introduction and Objectives

In this lab I implemented Ansible on Amazon EC2 to automate software installation on several servers. I created one central (control) node, **EC2-Central**, and three managed nodes, **M1**, **M2** and **M3**. All four servers run Ubuntu on `t3.micro` instances in the AWS `eu-north-1` region. Ansible was installed on the central node, which then connected to the managed nodes over SSH and configured them with an inventory file and a playbook.

In real-world DevOps work, one control node is used to configure many servers at the same time. This avoids logging in to every server by hand and makes sure each server ends up with exactly the same tools.

Objectives

- Launch one central EC2 instance and three managed EC2 instances with suitable security groups.
- Install Ansible on the central node and set up SSH key access to the managed nodes.
- Create an inventory file and test connectivity with the Ansible `ping` module.
- Install Docker and Visual Studio Code (VS Code) on the central node.
- Write and run a playbook, `install_tools.yml`, that installs Docker and VS Code on M1, M2 and M3.
- Verify the installations on the managed nodes from the central node.

2. Lab Environment

The table below lists the four servers used in this lab. The instance IDs and IP addresses were taken from the AWS console screenshots shown later in this report.

Node	Role	Instance ID	Public IPv4	Private IPv4
EC2-Central	Central (control) node	i-0aa2fb7ddc1deaca8	13.60.237.46	172.31.31.145
M1	Managed node	i-0eb6f5b591f7a03ad	51.20.136.251	172.31.23.4
M2	Managed node	i-0148a111312db5036	13.61.151.25	172.31.23.99
M3	Managed node	i-0ab1163753f2e5f9c	13.60.2.135	172.31.17.221

Common settings

- **Operating system:** Canonical Ubuntu 26.04 (amd64), AMI `ami-0aba19e56f3eaec05`.
- **Instance type and storage:** `t3.micro` with one 8 GiB volume.
- **Region and network:** `eu-north-1` (Availability Zone `eu-north-1a`), default VPC `vpc-0910ded3d03cc54ad` (`172.31.0.0/16`).
- **Key pair:** `centralserverkey`.
- **Security groups:** `SG-Ansible-Central` for the central node and `SG-Ansible-Managed` for the managed nodes.

3. Launching the AWS EC2 Instances

3.1 Launching the Central Node

I started in the EC2 console and chose **Launch instance**. The first server has to act as the Ansible control node, so I gave it a clear name and selected the Ubuntu image from the Quick Start list.

Settings used

- **Name:** `EC2-Central`
- **AMI:** Canonical Ubuntu 26.04 amd64 (`ami-0aba19e56f3eaec05`)
- **Instance type:** `t3.micro`
- **Number of instances:** 1
- **Storage:** 1 volume of 8 GiB

Result

The Summary panel on the right of the page confirms the AMI, the instance type, the new security group and the storage before the instance is launched.

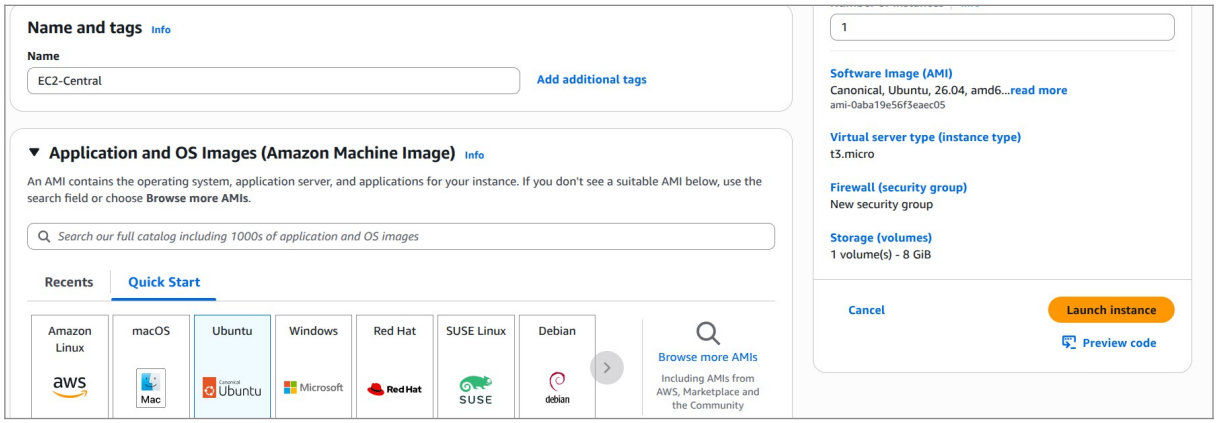


Figure 1: *Launch instance page for EC2-Central showing the name, the Ubuntu AMI and the instance summary.*

3.2 Configuring the Security Group and Network of the Central Node

Before launching, I created a new security group for the central node. A security group works as a virtual firewall, so the rules I set here decide who can reach the server.

Settings used

- **Security group name:** `SG-Ansible-Central`
- **Inbound rule 1:** SSH, TCP, port 22, source type *My IP* (`154.81.230.189/32`). SSH is therefore allowed only from my own public IP address.
- **Inbound rule 2:** TCP port 443 from `0.0.0.0/0`. Only the rule heading is visible at the bottom of Figure 2.
- **Network:** default VPC `vpc-0910ded3d03cc54ad` (`172.31.0.0/16`), subnet and Availability Zone set to *No preference*.
- **Auto-assign public IP:** Enable, so that I can connect to the server over the internet.

Result

Figure 2 shows the SSH rule restricted to my IP address. Figure 3 shows the network settings with *Create security group* selected and the group name `SG-Ansible-Central`.

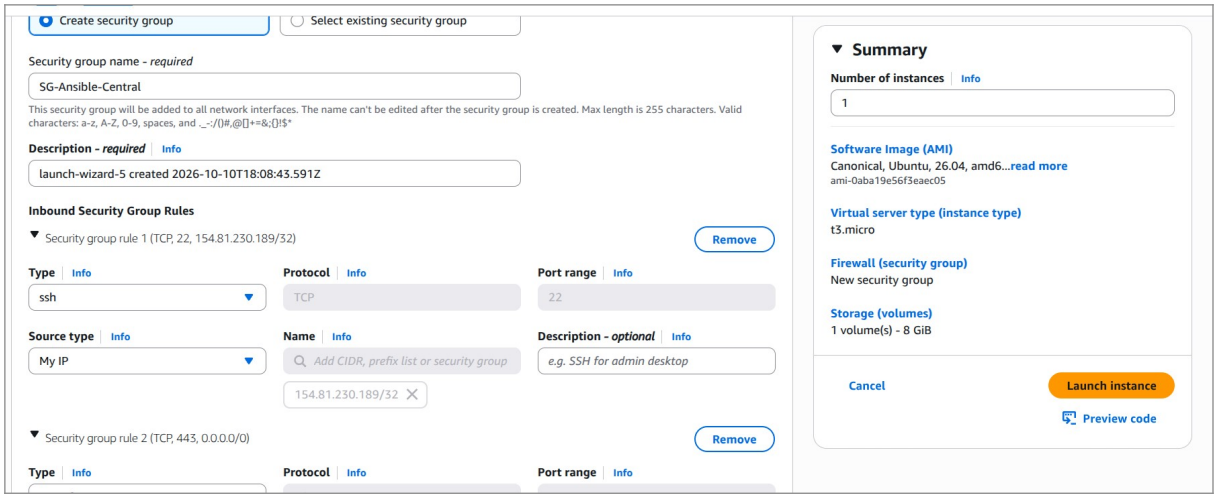


Figure 2: *Inbound security group rules for SG-Ansible-Central, with SSH limited to my IP address.*

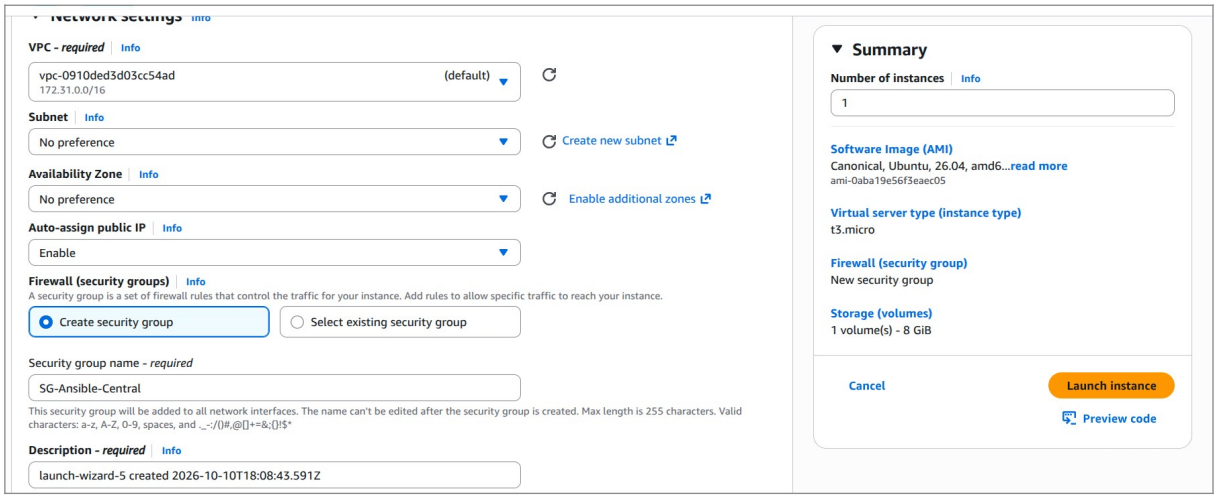


Figure 3: *Network settings and firewall configuration for the central node.*

3.3 Checking the Running Central Instance

After launching, I opened the instance details page to confirm that EC2-Central was running and to note its IP addresses. The public address is used for logging in from my computer, and the private address is used for communication inside the VPC.

Result

The page shows instance `i-0aa2fb7ddc1deaca8` (EC2-Central) in the **Running** state. Its public IPv4 address is `13.60.237.46`, its private IPv4 address is `172.31.31.145`, and its type is `t3.micro` in VPC `vpc-0910ded3d03cc54ad`.

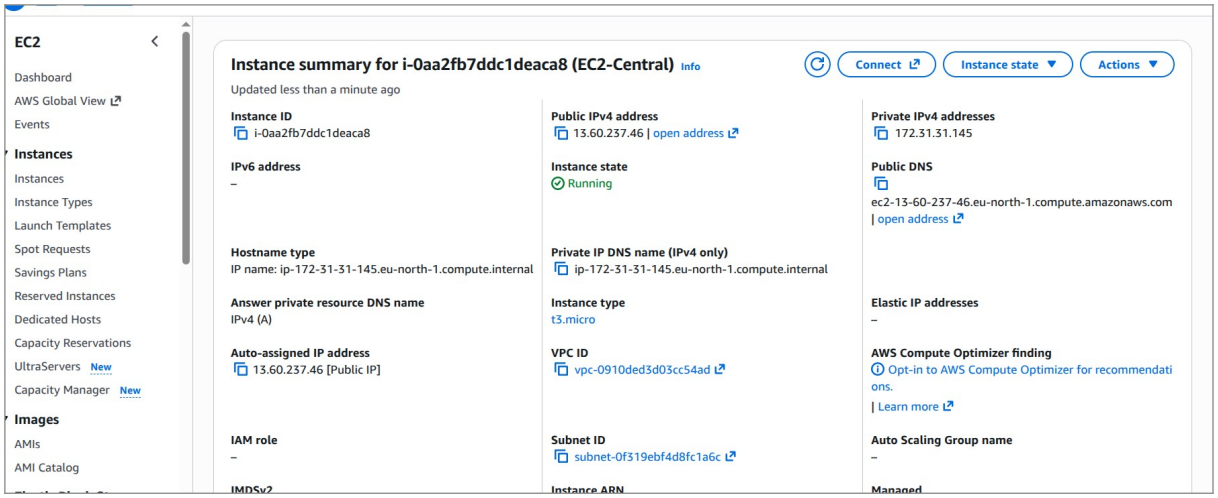


Figure 4: *Instance summary of EC2-Central in the Running state with its public and private IPv4 addresses.*

3.4 Launching the Managed Nodes

Next I launched the three managed nodes in one go. I used the same Ubuntu AMI and `t3.micro` type, but set the **Number of instances** to 3 so that AWS created three identical servers in a single launch. The three servers appear in the console as M1, M2 and M3 (Figure 7).

Settings used

- **Name:** `M1`
- **Number of instances:** 3
- **AMI and type:** Canonical Ubuntu 26.04 amd64, `t3.micro`
- **Storage:** 1 volume of 8 GiB

Result

The Summary panel shows 3 instances with the same AMI, instance type and storage as the central node. AWS also suggests EC2 Auto Scaling for launches of more than one instance, which is not needed in this lab.

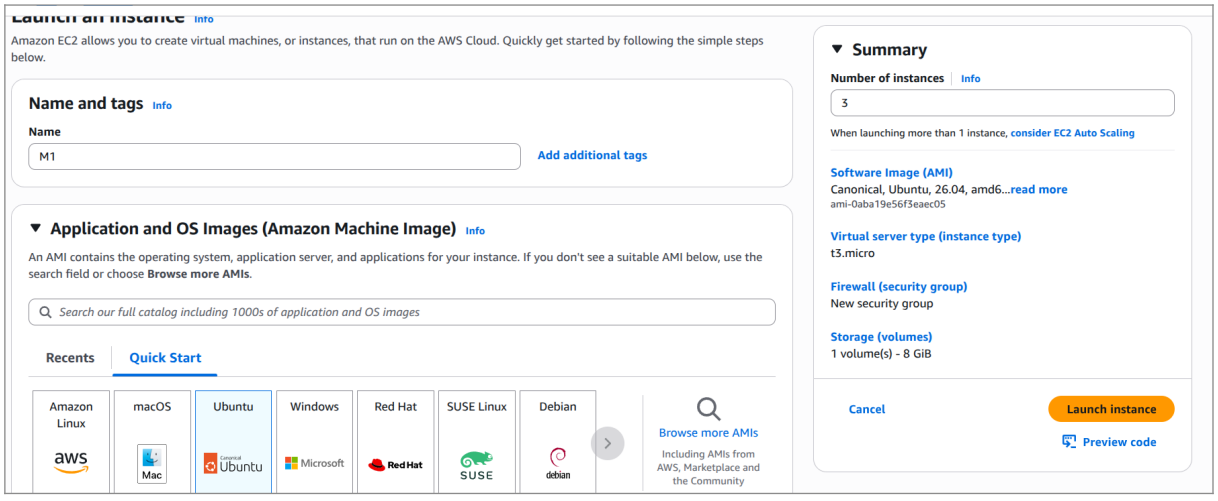


Figure 5: *Launch instance page for the managed nodes with the number of instances set to 3.*

3.5 Configuring the Network and Security Group of the Managed Nodes

The managed nodes get their own security group, `SG-Ansible-Managed`, so that their firewall rules are separate from those of the central node. Its description states the purpose: allow SSH access from the central node for Ansible management.

Settings used

- **VPC:** `vpc-0910ded3d03cc54ad` (default), `172.31.0.0/16`
- **Subnet:** `subnet-0f319ebf4d8fc1a6c` in `eu-north-1a`, CIDR `172.31.16.0/20`
- **Auto-assign public IP:** Enable
- **Security group name:** `SG-Ansible-Managed`
- **Description:** Allow SSH access from EC2 Central for Ansible management

Result

The screenshot shows the selected subnet, public IP assignment enabled, and a new security group named `SG-Ansible-Managed` ready to be created with the launch.

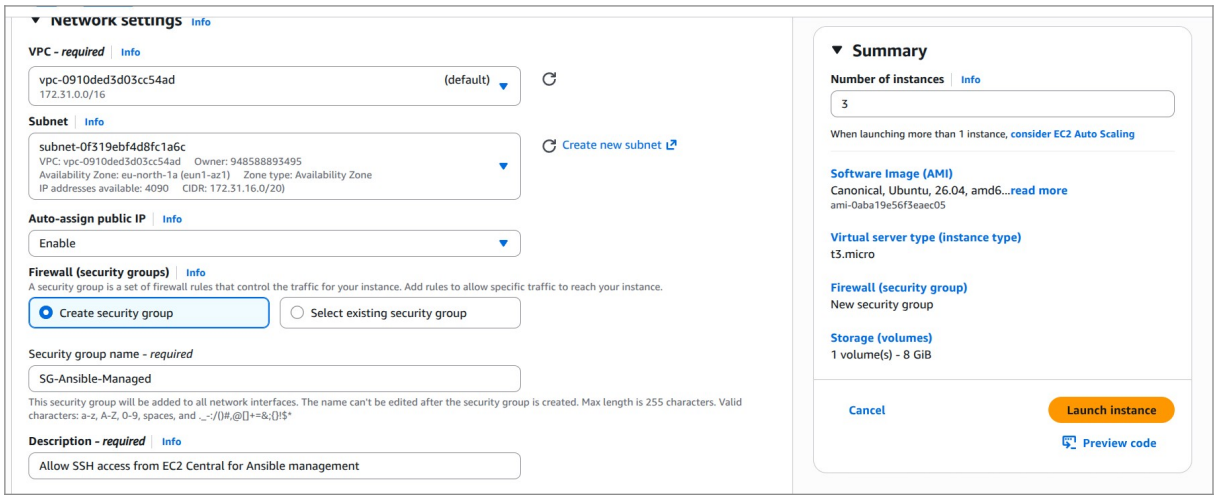


Figure 6: *Network settings and the SG-Ansible-Managed security group for the managed nodes.*

3.6 Confirming That All Four Instances Are Running

After the launch I opened the **Instances** list to check the state of all four servers, and then scrolled the table to the right to see their security groups, key pair and launch time.

Result

In Figure 7, EC2-Central, M1, M2 and M3 are all in the **Running** state, all are `t3.micro` in `eu-north-1a`. M2 and M3 have already passed their 3/3 status checks, while M1 still shows *Initializing*. In Figure 8, EC2-Central uses `SG-Ansible-Central`, the three managed nodes use `SG-Ansible-Managed`, and all four use the key pair `centralserverkey`. The central node was launched at 23:22 and the managed nodes at 23:38 (GMT+5).

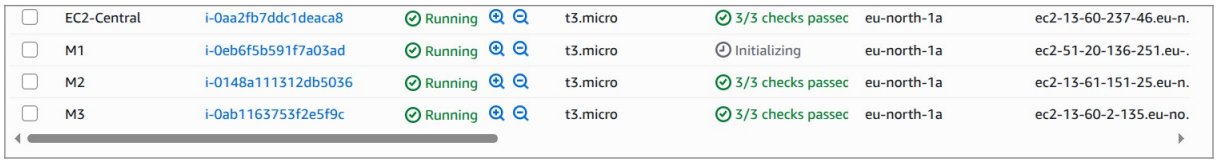


Figure 7: *Instances list showing EC2-Central, M1, M2 and M3 in the Running state.*

sabled	SG-Ansible-Central	centralserverkey	2026/10/10 23:22 GMT+5	Linux/UNIX	false	–
sabled	SG-Ansible-Managed	centralserverkey	2026/10/10 23:38 GMT+5	Linux/UNIX	false	–
sabled	SG-Ansible-Managed	centralserverkey	2026/10/10 23:38 GMT+5	Linux/UNIX	false	–
sabled	SG-Ansible-Managed	centralserverkey	2026/10/10 23:38 GMT+5	Linux/UNIX	false	–

Figure 8: *Security group, key pair and launch time of the four instances.*

3.7 Allowing SSH Only from the Central Node

The managed nodes should accept SSH connections from the central node only, not from the whole internet. To do this I opened **Edit inbound rules** for the security group `sg-08bb9445dd42f8c81` (`SG-Ansible-Managed`) and defined the SSH rule with another security group as its source.

Settings used

- **Type:** SSH (TCP, port 22)
- **Source type:** Custom
- **Source:** security group `sg-063a23175c9ce7764`

Result

Because the source is a security group and not an IP address range, only instances that belong to that group, here the central node, can open SSH connections to the managed nodes. Figure 9 shows the rule before saving. Figure 10 shows the **Security** tab of M1 (`i-0eb6f5b591f7a03ad`) afterwards: it uses `sg-08bb9445dd42f8c81` (`SG-Ansible-Managed`) and lists inbound rule `sgr-0f727592658c06cfd` for TCP port 22 with source `sg-063a23175c9ce7764`.

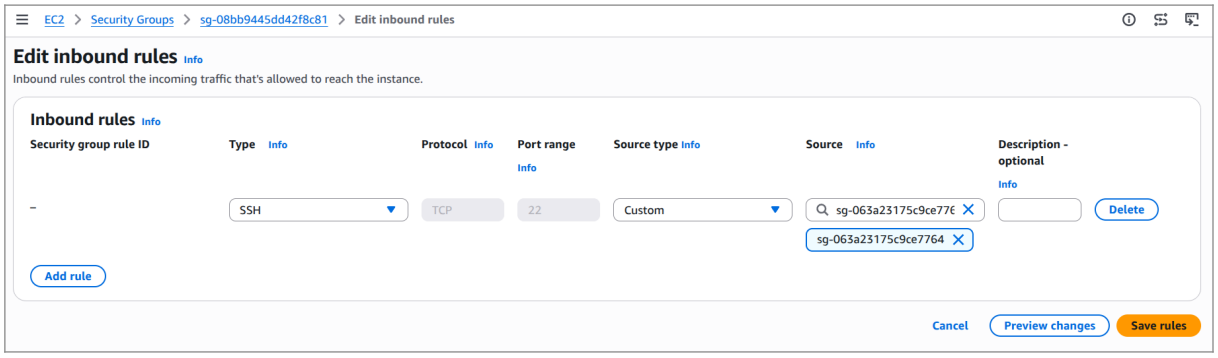


Figure 9: *Editing the inbound SSH rule of SG-Ansible-Managed so that the source is a security group.*

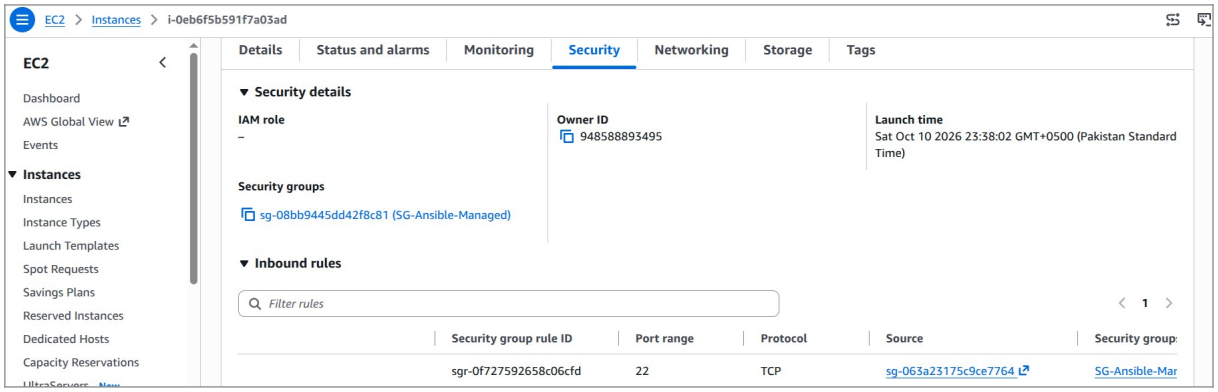


Figure 10: *Security tab of M1 confirming the inbound SSH rule on port 22.*

4. Connecting to the Central Node and Installing Ansible

4.1 Logging in to the Central Node over SSH

With the instances running, I logged in to the central node from my own computer. This gives me a terminal on the EC2 server where Ansible will be installed.

Command used

```
ssh -i centralserverkey.pem ubuntu@13.60.237.46
```

The title bar of the terminal in Figure 11 shows `ssh -i centralserverkey.pem ubuntu...`. The address `13.60.237.46` is the public IPv4 address of EC2-Central from Figure 4.

Command explanation

- `ssh` starts a secure shell session with a remote server.
- `-i centralserverkey.pem` selects the private key file used for authentication.
- `ubuntu@13.60.237.46` gives the login user (`ubuntu`, the default user of Ubuntu AMIs) and the server address.

Result

The login succeeded. The Ubuntu welcome message reports 25% memory usage, 0 users logged in and the address `172.31.31.145` for interface `ens5`, and the prompt changes to `ubuntu@ip-172-31-31-145`, so I am now working on the central node.

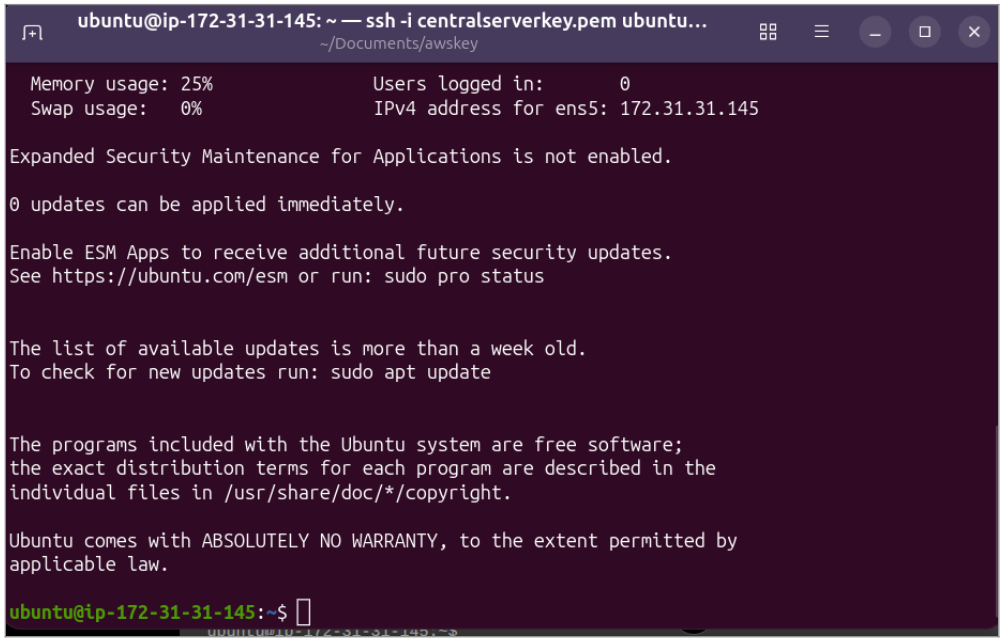


Figure 11: *SSH login to the central node (EC2-Central).*

4.2 Installing Ansible

Ansible is the automation tool of this lab, so I installed it on the central node with the Ubuntu package manager.

Command used

```
sudo apt install -y ansible
```

Command explanation

- `sudo` runs the command with administrator rights.
- `apt install` installs a package from the Ubuntu repositories.
- `-y` answers *yes* to the confirmation prompt automatically.
- `ansible` is the package to install.

Result

The output lists `ansible` together with its dependencies, such as `ansible-core` and several `python3-*` packages. The summary reports 14 packages to be installed with a download size of 22.4 MB. The installation is confirmed later by `ansible --version` (Figure 39).

```
ubuntu@ip-172-31-31-145:~$ sudo apt install -y ansible
Installing:
  ansible

Installing dependencies:
  ansible-core         python3-gssapi    python3-requests-ntlm  python3-xmltodict
  python3-argcomplete  python3-kerberos  python3-resolvelib
  python3-decorator    python3-libcloud  python3-selinux
  python3-dnspython    python3-pyspnego  python3-winrm

Suggested packages:
  cowsay   python3-aioquic  python3-httpcore  python3-trio
  sshpass  python3-h2       python3-httpx

Summary:
  Upgrading: 0, Installing: 14, Removing: 0, Not Upgrading: 189
  Download size: 22.4 MB
```

Figure 12: *Installing Ansible on the central node using apt.*

4.3 Copying the Key Pair to the Central Node

Ansible on the central node has to log in to the managed nodes, so it needs the private key too. I ran the following command in a terminal on my own computer (prompt `sami@sami`, folder `~/Documents/awskey`), not on the server.

Command used

```
scp -i centralserverkey.pem centralserverkey.pem \
    ubuntu@13.60.237.46:/home/ubuntu/ansible-lab.pem
```

Command explanation

- `scp` copies files securely between computers over SSH.
- `-i centralserverkey.pem` is the key used to log in to the server for the transfer.
- `centralserverkey.pem` (first occurrence after the option) is the local file being copied.
- `ubuntu@13.60.237.46:/home/ubuntu/ansible-lab.pem` is the destination: the central node, saved as `ansible-lab.pem` in the home folder.

Result

The transfer line shows 100% completed for a file of 1678 bytes. The key is now on the central node as `/home/ubuntu/ansible-lab.pem`, which is the path I later use in the Ansible inventory.

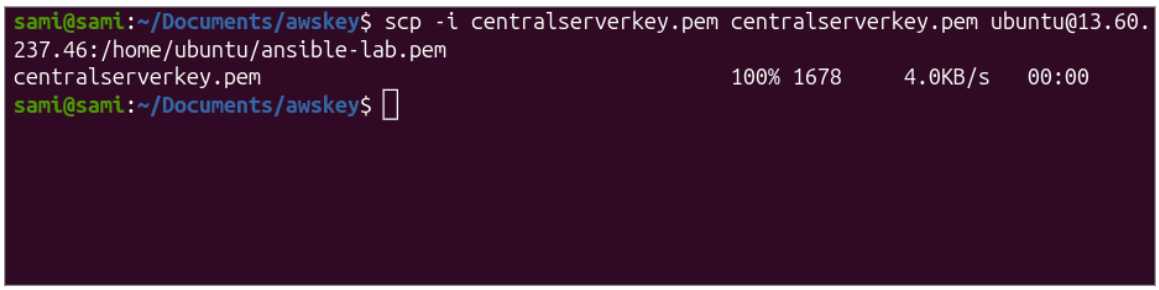


Figure 13: *Copying the key pair to the central node using scp.*

4.4 Protecting the Private Key

SSH refuses to use a private key that other users can read. I logged in to the central node again and restricted the permissions of the copied key to the owner only.

Commands used

```
chmod 600 ~/ansible-lab.pem
ls -l ~/ansible-lab.pem
```

Command explanation

- `chmod 600` sets the permissions so that only the owner can read and write the file.
- `~/ansible-lab.pem` is the key file in my home directory.
- `ls -l` lists the file in long format so the permissions can be checked.

Result

The listing shows `-rw-------` for `/home/ubuntu/ansible-lab.pem`, owned by `ubuntu` with a size of 1678 bytes, so the permissions are correct. The login message at the top (*Last login ... from 154.81.230.189*) shows this is a new session.

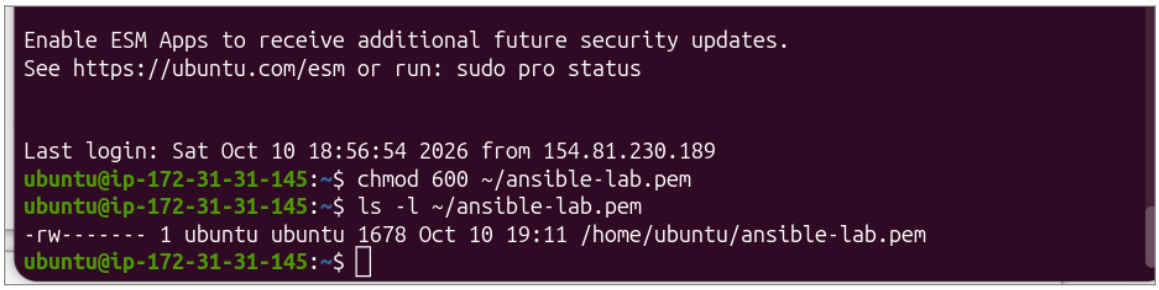


Figure 14: *Setting and verifying the permissions of ansible-lab.pem.*

5. Verifying SSH Access to the Managed Nodes

Before using Ansible, I checked that I could log in to each managed node. For every node I first noted its IP addresses on the EC2 instance summary page and then opened an SSH session to it using the key `ansible-lab.pem` and its private address. This is the same connection path Ansible uses later. The terminal prompt shows which server I am on, so it serves as proof of the login.

5.1 Managed Node M1

Result

Figure 15 shows M1 (`i-0eb6f5b591f7a03ad`) running with public IPv4 `51.20.136.251` and private IPv4 `172.31.23.4`. In Figure 16 the prompt reads `ubuntu@ip-172-31-23-4`, which matches the private IP of M1, so the login worked.

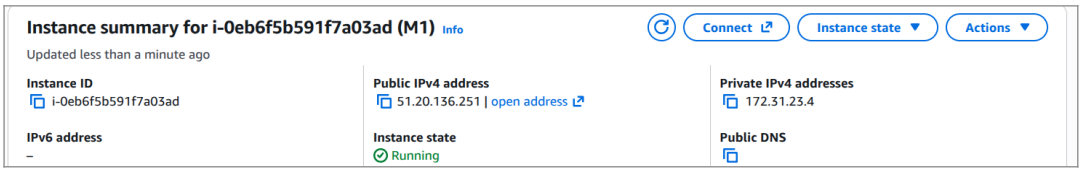


Figure 15: *Instance summary of M1 showing its public and private IP addresses.*

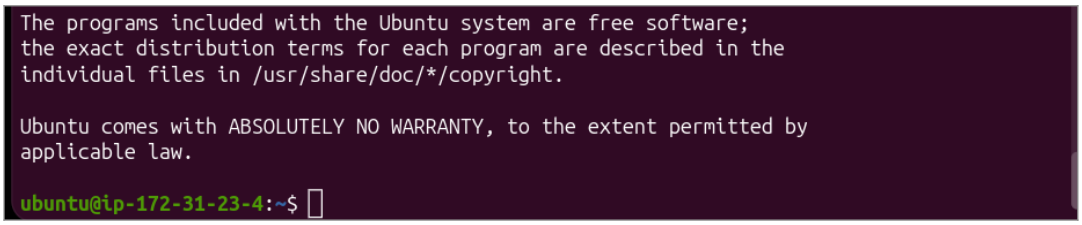


Figure 16: *SSH session on M1 (prompt ip-172-31-23-4).*

5.2 Managed Node M2

Result

Figure 17 shows M2 (`i-0148a111312db5036`) running with public IPv4 `13.61.151.25` and private IPv4 `172.31.23.99`. In Figure 18 the prompt reads `ubuntu@ip-172-31-23-99`, which confirms the login to M2.

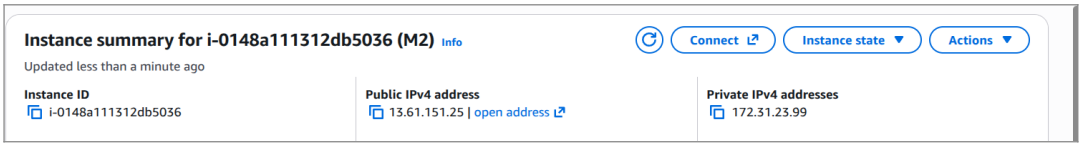


Figure 17: *Instance summary of M2 showing its public and private IP addresses.*

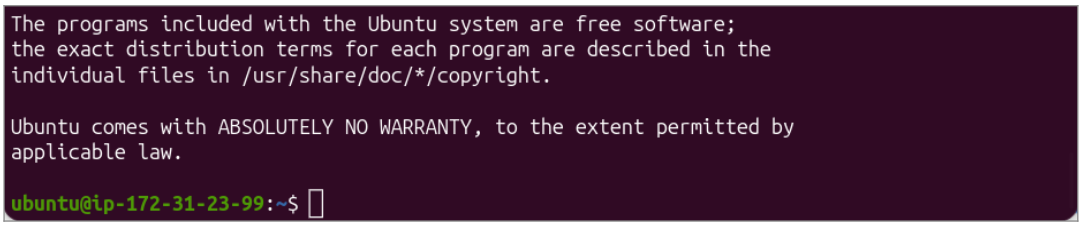


Figure 18: *SSH session on M2 (prompt ip-172-31-23-99).*

5.3 Managed Node M3

Result

Figure 19 shows M3 (`i-0ab1163753f2e5f9c`) with public IPv4 `13.60.2.135` and private IPv4 `172.31.17.221`. In Figure 20 the prompt reads `ubuntu@ip-172-31-17-221`, which confirms the login to M3.

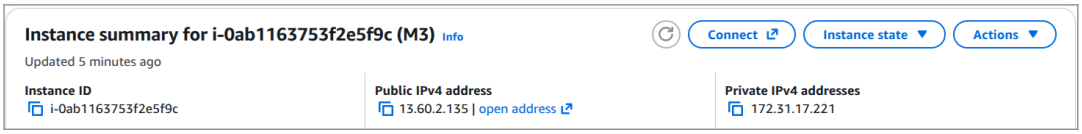


Figure 19: *Instance summary of M3 showing its public and private IP addresses.*

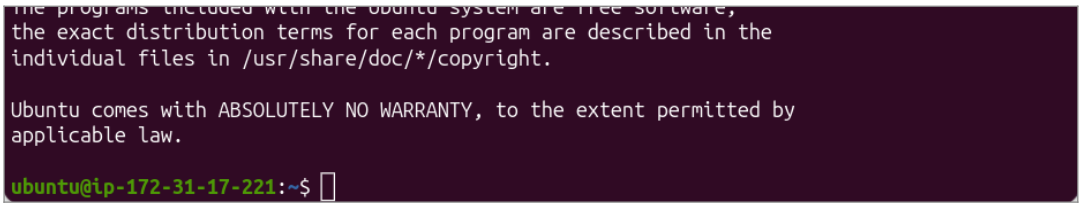


Figure 20: *SSH session on M3 (prompt ip-172-31-17-221).*

6. Creating the Inventory and Testing Ansible Connectivity

6.1 Inventory File and the Ansible Ping Test

Ansible needs an inventory file that tells it which servers to manage and how to connect to them. On the central node I created a project folder, wrote the inventory file with the private IP addresses of M1, M2 and M3, and then tested the connection to all three nodes with the `ping` module.

Commands used

```
mkdir -p ~/ansible-lab
cd ~/ansible-lab
nano inventory.ini
ansible -i inventory.ini managed -m ping
```

Command explanation

- `mkdir -p ~/ansible-lab` creates the folder `ansible-lab` in my home directory (`-p` avoids an error if it already exists).
- `cd ~/ansible-lab` moves into that folder.
- `nano inventory.ini` opens the nano text editor to create the inventory file.
- `ansible -i inventory.ini managed -m ping` runs an ad hoc command: `-i` selects the inventory file, `managed` is the host group, and `-m ping` runs the `ping` module.

What the inventory contains

- The group **[managed]** lists the three hosts: `M1 ansible_host=172.31.23.4`, `M2 ansible_host=172.31.23.99` and `M3 ansible_host=172.31.17.221`.
- The group **[managed:vars]** sets variables for all of them: `ansible_user=ubuntu`, `ansible_ssh_private_key_file=/home/ubuntu/ansible-lab.pem` and `ansible_python_interpreter=/usr/bin/python3`.

Result

Figure 21 shows the inventory file in nano. In Figure 22 the three commands appear at the top; the first line, *Connection to 172.31.17.221 closed.*, is left over from my earlier SSH session to M3. The ping test returns `SUCCESS` for M2, M3 and M1, each with `"changed": false` and `"ping": "pong"`. This `ping` is not a network ping: it confirms that Ansible can log in over SSH and run Python on each node, so the central node is ready to manage all three servers.



Figure 21: *Contents of inventory.ini in the nano editor.*

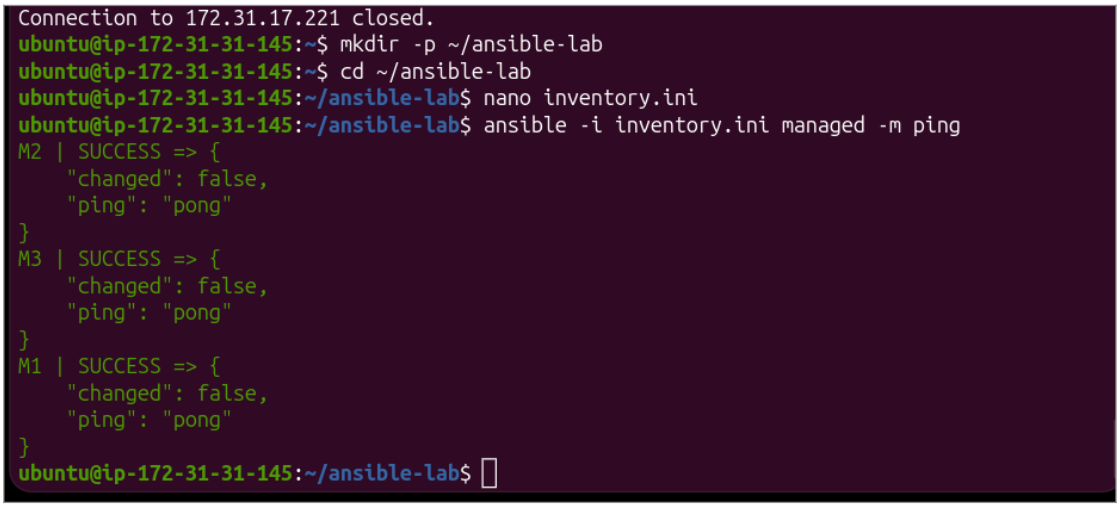


Figure 22: *Creating the project folder and testing connectivity with the Ansible ping module.*

7. Installing Docker on the Central Node

7.1 Updating the Package Index

First I prepared the central node for the Docker installation by refreshing the package list and installing two helper packages that are needed to download and verify Docker's repository.

Commands used

```
sudo apt update
sudo apt install -y ca-certificates curl
```

Command explanation

- `sudo apt update` refreshes the list of available packages from the configured repositories.
- `ca-certificates` provides trusted certificates for HTTPS connections.
- `curl` is a tool for downloading files from the command line.

Result

The `Hit:` lines show that the Ubuntu repositories were reachable, and the output notes that 189 packages can be upgraded. The install step upgrades four packages (`ca-certificates`, `curl`, `libcurl3t64-gnutls` and `libcurl4t64`) with a download size of 1258 kB.

```
ubuntu@ip-172-31-31-145:~/ansible-lab$ sudo apt update
sudo apt install -y ca-certificates curl
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
189 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  ca-certificates  curl  libcurl3t64-gnutls  libcurl4t64

Summary:
  Upgrading: 4, Installing: 0, Removing: 0, Not Upgrading: 185
  Download size: 1258 kB
  Freed space: 31.7 kB
```

Figure 23: *Updating the package index and installing ca-certificates and curl on the central node.*

7.2 Adding the Official Docker Repository

Docker is not installed from the default Ubuntu repositories in this lab. Instead, I added the official Docker repository together with its signing key, so that apt can download and verify Docker packages directly from Docker.

Commands used

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo ${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

In the screenshot the long `echo` command wraps over several lines on screen; it is one command, shown above with line breaks (`\`) added for readability.

Command explanation

- `install -m 0755 -d /etc/apt/keyrings` creates the keyrings directory with permissions 755.
- `curl -fsSL … -o /etc/apt/keyrings/docker.asc` downloads Docker's GPG key. `-f` fails on server errors, `-s` is silent, `-S` still shows errors, `-L` follows redirects and `-o` sets the output file.
- `chmod a+r` makes the key readable for all users so that apt can use it.
- `echo "deb [...] stable" | sudo tee /etc/apt/sources.list.d/docker.list` writes the repository line into a new apt source file. It uses the system architecture (`dpkg --print-architecture`), the downloaded key (`signed-by=`), and the Ubuntu codename read from `/etc/os-release`. `> /dev/null` hides the echoed output.

Result

All commands completed without error messages and the prompt returned each time, so the key and the repository file were created.

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-31-145:~/ansible-lab$ sudo install -m 0755 -d /etc/apt/keyrings
ubuntu@ip-172-31-31-145:~/ansible-lab$ sudo curl -fsSL https://download.docker.com/linux/ubu
ntu/gpg \
  -o /etc/apt/keyrings/docker.asc
ubuntu@ip-172-31-31-145:~/ansible-lab$ sudo chmod a+r /etc/apt/keyrings/docker.asc
ubuntu@ip-172-31-31-145:~/ansible-lab$ echo "deb [arch=$(dpkg --print-architecture) signed-b
y=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release
 && echo ${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
ubuntu@ip-172-31-31-145:~/ansible-lab$
```

Figure 24: *Adding Docker's GPG key and apt repository on the central node.*

7.3 Installing Docker Engine

With the repository configured, I installed Docker Engine and its related components in one command.

Command used

```
sudo apt install -y docker-ce docker-ce-cli \
  containerd.io docker-buildx-plugin docker-compose-plugin
```

Command explanation

- `docker-ce` is the Docker Engine (daemon) and `docker-ce-cli` is the `docker` command-line client.
- `containerd.io` is the container runtime used by Docker.
- `docker-buildx-plugin` and `docker-compose-plugin` add the `docker buildx` and `docker compose` commands.

Result

Apt lists the five requested packages plus the dependencies `docker-ce-rootless-extras` and `pigz`. The summary says 7 packages will be installed, with a 102 MB download and 388 MB of disk space needed out of 4093 MB available. The last line shows the download starting.

```
ubuntu@ip-172-31-31-145:~/ansible-lab$ sudo apt install -y docker-ce docker-ce-cli \
  containerd.io docker-buildx-plugin docker-compose-plugin
Installing:
  containerd.io  docker-buildx-plugin  docker-ce  docker-ce-cli  docker-compose-plugin

Installing dependencies:
  docker-ce-rootless-extras  pigz

Suggested packages:
  cgroupfs-mount  | cgroup-lite  docker-model-plugin

Summary:
  Upgrading: 0, Installing: 7, Removing: 0, Not Upgrading: 185
  Download size: 102 MB
  Space needed: 388 MB / 4093 MB available

Get:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 pigz amd64 2
```

Figure 25: *Installing Docker Engine and its plugins with apt.*

7.4 Verifying Docker with the hello-world Container

To make sure Docker works, I ran the small test image `hello-world`. This checks the whole chain: client, daemon, image download and container start.

Command used

```
sudo docker run hello-world
```

Command explanation

- `docker run` creates and starts a container from an image.
- `hello-world` is a tiny official image that prints a test message and exits.

Result

Docker reports *Unable to find image 'hello-world:latest' locally*, pulls it from `library/hello-world`, and the status says a newer image was downloaded. The container then prints *Hello from Docker!* and the message that the installation appears to be working correctly, followed by the steps Docker took.

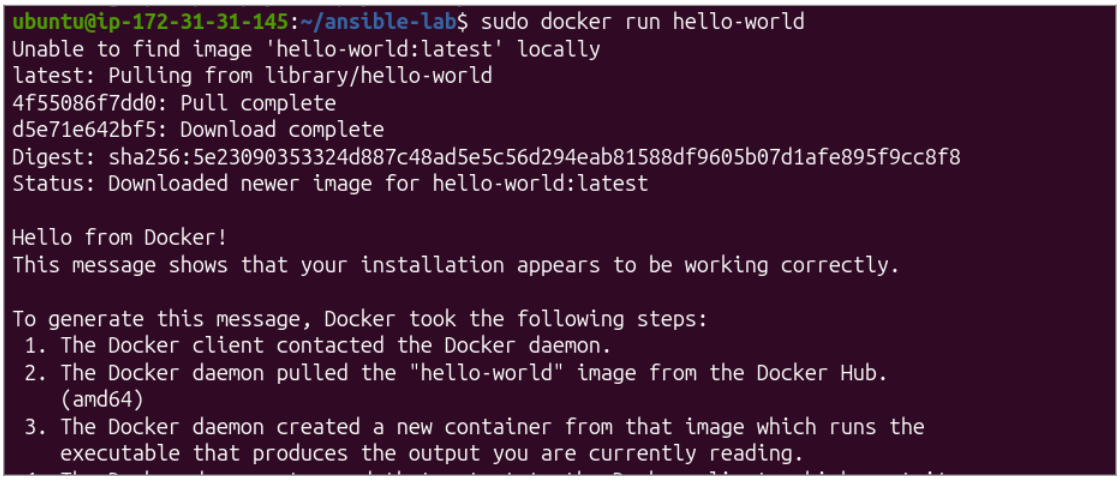


Figure 26: *Verifying the Docker installation by running the hello-world container.*

8. Installing VS Code on the Central Node

8.1 Downloading and Installing VS Code

Next I installed Visual Studio Code on the central node by downloading the official `.deb` package into the `/tmp` folder and installing it with apt.

Commands used

```
cd /tmp
curl -L \
  'https://update.code.visualstudio.com/latest/linux-deb-x64/stable' \
  -o code.deb
sudo apt install -y ./code.deb
```

Command explanation

- `cd /tmp` changes to the temporary folder so the installer is not saved in the project directory.
- `curl -L … -o code.deb` downloads the latest 64-bit Linux VS Code package from Microsoft and saves it as `code.deb`; `-L` follows redirects.
- `sudo apt install -y ./code.deb` installs the local package with apt, which also installs the dependencies it needs.

Result

The curl progress table shows the 237.6 MB package was downloaded in about one second. Apt notes *selecting 'code' instead of './code.deb'*, lists `code` as the package to install, and begins listing its dependencies.

```
ubuntu@ip-172-31-31-145:~/ansible-lab$ cd /tmp

curl -L \
  'https://update.code.visualstudio.com/latest/linux-deb-x64/stable' \
  -o code.deb
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
100    163 100    163   0      0   2283      0                              0
100 237.6M 100 237.6M   0      0 154.2M      0   00:01   00:01         172.4M
ubuntu@ip-172-31-31-145:/tmp$ sudo apt install -y ./code.deb
Note, selecting 'code' instead of './code.deb'
Installing:
  code

Installing dependencies:
  adwaita-icon-theme          libfile-desktopentry-perl  libwayland-egl1
  alacritty                   libfile-mimeinfo-perl      libx11-protocol-perl
  alsa-topology-conf          libfontconfig1             libx11-xcb1
  alsa-ucm-conf               libgbm1                    libxaw7
  at-spi2-common              libgdk-pixbuf-2.0-0        libxcb-dri3-0
```

Figure 27: *Downloading and installing the VS Code .deb package on the central node.*

8.2 Checking the VS Code Version

Finally, I verified the installation by asking VS Code for its version.

Command used

```
code --version
```

Command explanation

`code` starts VS Code, and `--version` makes it print the version information and exit instead of opening the editor.

Result

The output shows version `1.141.0`, the commit ID `2a59476c9bfcb90b3ddc372c36762471b7dfad1c` and the architecture `x64`, so VS Code is installed on the central node.

```
ubuntu@ip-172-31-31-145:/tmp$ code --version
1.141.0
2a59476c9bfcb90b3ddc372c36762471b7dfad1c
x64
ubuntu@ip-172-31-31-145:/tmp$
```

Figure 28: *Checking the installed VS Code version on the central node.*

9. Writing the Ansible Playbook

To install the same tools on M1, M2 and M3 without repeating every step by hand, I wrote a playbook named `install_tools.yml` in the `~/ansible-lab` folder. A playbook is a YAML file that describes the desired state of the servers as a list of tasks.

Command used

```
nano install_tools.yml
```

Command explanation

`nano` opens the text editor. The editor header in Figure 29 shows the file name `install_tools.yml`; the asterisk means the file has not been saved yet.

What the playbook contains

- `name: Install Docker and VS Code on managed instances` is the title of the play.
- `hosts: managed` runs the play on the `managed` group from the inventory (M1, M2, M3).
- `become: true` runs the tasks with administrator (sudo) rights.
- `gather_facts: true` collects information about each node before the tasks run.
- Task **Install required packages** uses `ansible.builtin.apt` to make sure `ca-certificates` and `curl` are present (`state: present`) and refreshes the package cache (`update_cache: true`).
- Task **Create Docker keyrings directory** uses `ansible.builtin.file` to create `/etc/apt/keyrings` (`state: directory`).

Result

Figure 29 shows the beginning of the playbook. The remaining tasks, such as downloading the Docker key, configuring the repository, installing Docker and setting up VS Code, appear by name in the playbook output in Section 10.

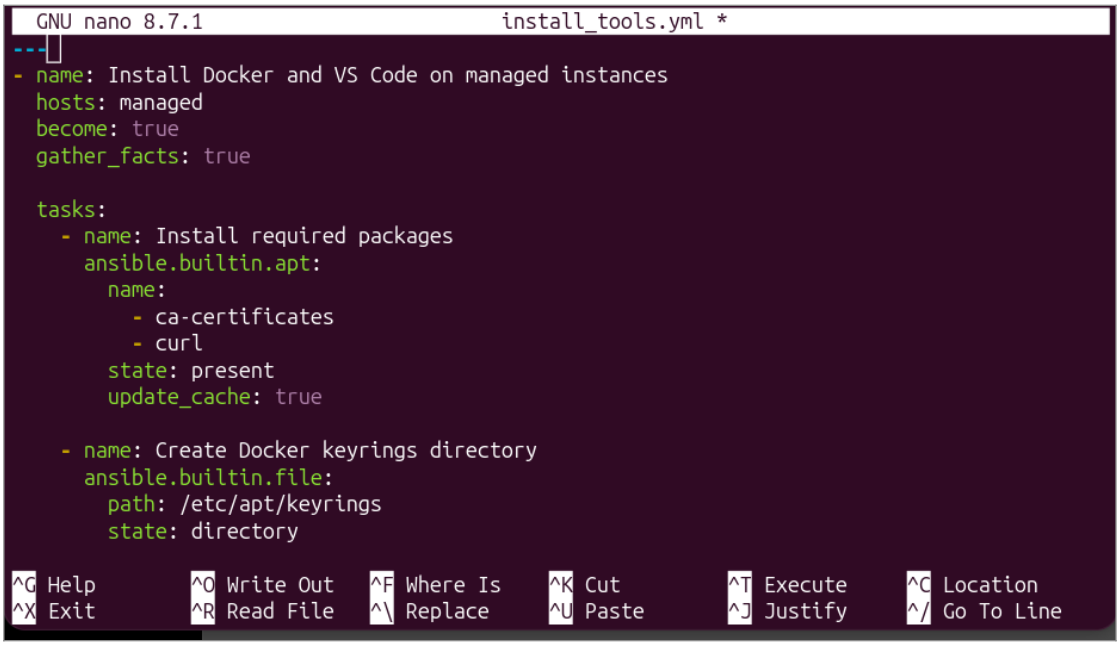


Figure 29: *First part of the install_tools.yml playbook in the nano editor.*

10. Running the Playbook on the Managed Nodes

10.1 Syntax Check and Start of the Run

Before running the playbook, I checked it for syntax errors. Then I ran it against the managed nodes so that Ansible would carry out every task on M1, M2 and M3.

Commands used

```
ansible-playbook -i inventory.ini install_tools.yml --syntax-check
ansible-playbook -i inventory.ini install_tools.yml
```

Command explanation

- `ansible-playbook` runs a playbook.
- `-i inventory.ini` tells it which servers to use.
- `install_tools.yml` is the playbook file.
- `--syntax-check` only checks that the playbook is valid YAML and valid Ansible, without running any task.

Result

The syntax check only prints `playbook: install_tools.yml` and no errors. The real run then starts with the play *Install Docker and VS Code on managed instances*. The **Gathering Facts** task returns `ok` for M2, M3 and M1, and the task **Install required packages** is just starting.

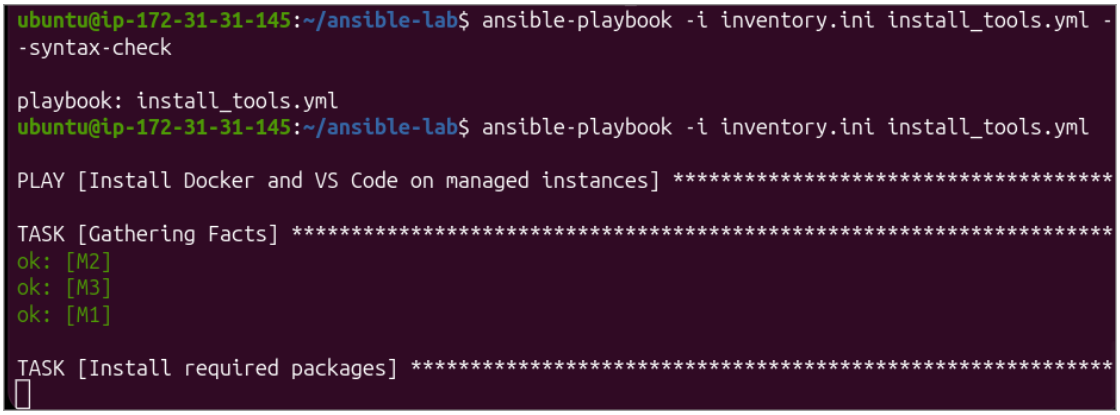


Figure 30: *Syntax check and start of the playbook execution.*

10.2 Package, Keyring and Signing Key Tasks

Ansible then works through the tasks in order and prints a status for every node. `ok` means the node already matched the desired state, `changed` means Ansible modified it, and `skipping` means the task was not run.

Result

Figure 31 shows **Install required packages** with `changed` on M3, M2 and M1, so the packages were installed or updated on all three nodes. **Create Docker keyrings directory** and **Download Docker signing key** are `ok` on all three nodes, and the task **Configure official Docker repository** is the next one starting.

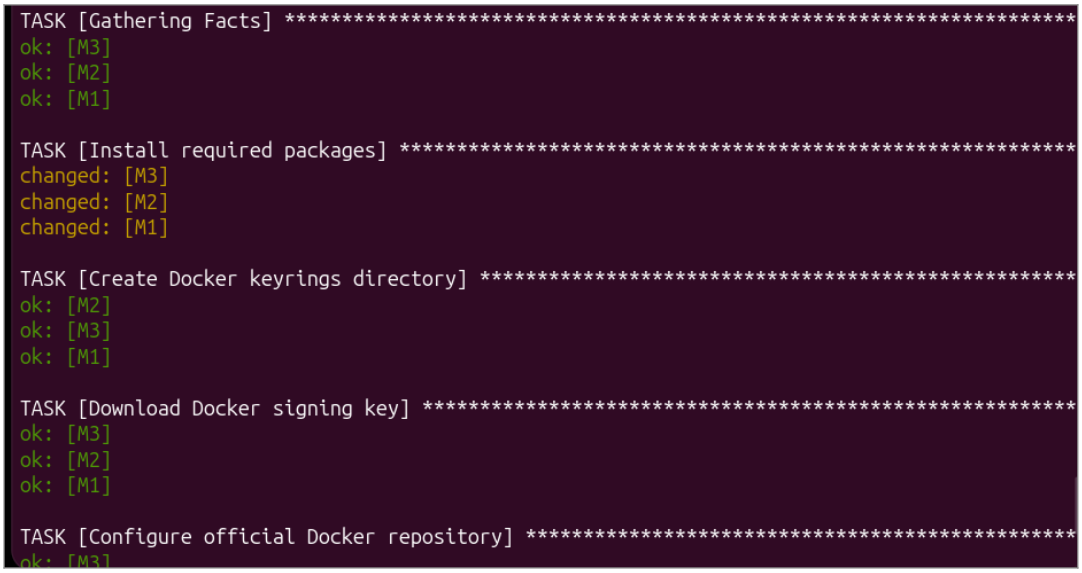


Figure 31: *Playbook output for the package, keyring directory and Docker signing key tasks.*

10.3 Docker Repository, Installation and Service Tasks

Result

In Figure 32 the task **Configure official Docker repository** is `ok` on M3, M2 and M1. The task **Install Docker Engine and plugins** is `ok` on all three nodes, followed by **Enable and start Docker**, which is also `ok` on all three, so Docker is installed and its service is set up on every managed node.

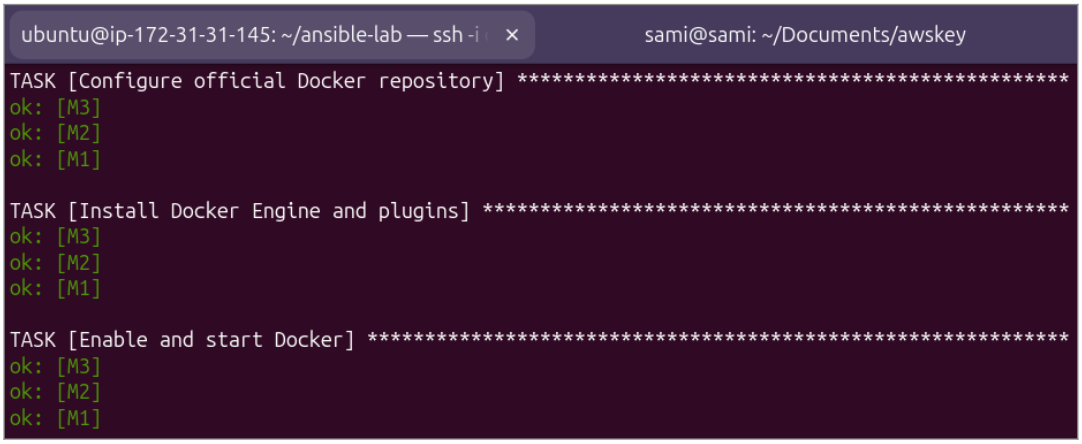


Figure 32: *Playbook output for the Docker repository, Docker Engine installation and Docker service tasks.*

10.4 VS Code Tasks

Result

Figure 33 shows the VS Code tasks. **Download VS Code Linux x64 archive** is `ok` on M1, M2 and M3. **Extract VS Code archive** is shown as `skipping` on all three nodes, meaning Ansible did not run that task on this run. **Set VS Code sandbox helper permissions** and **Create VS Code command link** are both `ok` on all three nodes.

```
TASK [Download VS Code Linux x64 archive] ***************************************************
ok: [M1]
ok: [M2]
ok: [M3]

TASK [Extract VS Code archive] **************************************************************
skipping: [M1]
skipping: [M2]
skipping: [M3]

TASK [Set VS Code sandbox helper permissions] ***********************************************
ok: [M1]
ok: [M2]
ok: [M3]

TASK [Create VS Code command link] **********************************************************
ok: [M1]
ok: [M2]
ok: [M3]
```

Figure 33: *Playbook output for the VS Code download, extraction, permission and command-link tasks.*

10.5 Verification Tasks

The last tasks of the playbook check the result of the installation on each node.

Result

Figure 34 shows **Verify Docker installation** as `ok` on all nodes, and **Display Docker version** prints `Docker version 29.9.0, build f415da8` for M1, M2 and M3. **Verify VS Code executable exists** is also `ok` on every node. Figure 35 shows **Confirm VS Code executable is installed**, which prints the message *VS Code executable is installed.* with `"changed": false` for M1, M2 and M3.

```
TASK [Verify Docker installation] ***********************************************************
ok: [M2]
ok: [M3]
ok: [M1]

TASK [Display Docker version] ****************************************************************
ok: [M1] => {
    "docker_version.stdout": "Docker version 29.9.0, build f415da8"
}
ok: [M2] => {
    "docker_version.stdout": "Docker version 29.9.0, build f415da8"
}
ok: [M3] => {
    "docker_version.stdout": "Docker version 29.9.0, build f415da8"
}

TASK [Verify VS Code executable exists] *****************************************************
ok: [M2]
ok: [M3]
ok: [M1]
```

Figure 34: *Playbook output for the Docker and VS Code verification tasks.*

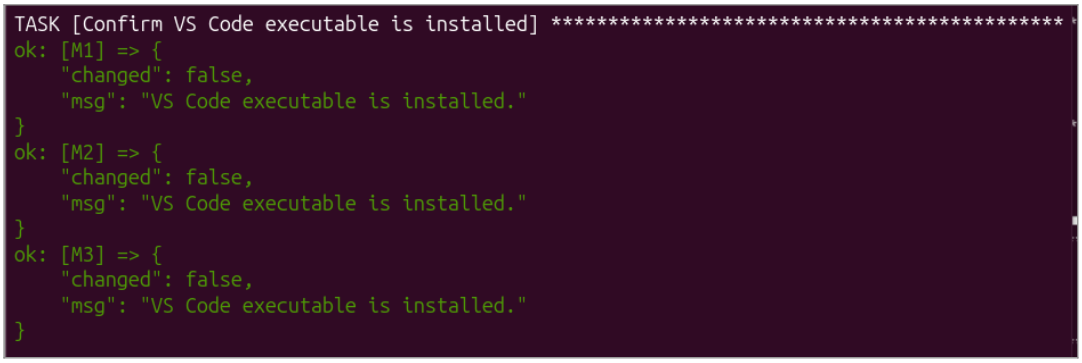


Figure 35: *Playbook output confirming that the VS Code executable is installed on all managed nodes.*

10.6 Play Recap

At the end of every run Ansible prints a PLAY RECAP, which summarises the results per node. This is the quickest way to see whether the play succeeded.

Result

For each of M1, M2 and M3 the recap shows `ok=14`, `changed=0`, `unreachable=0`, `failed=0`, `skipped=1`, `rescued=0` and `ignored=0`. No task failed and no node was unreachable. The single skipped task is the extraction task seen in Figure 33, and `changed=0` means Ansible found nothing further to change on this run.

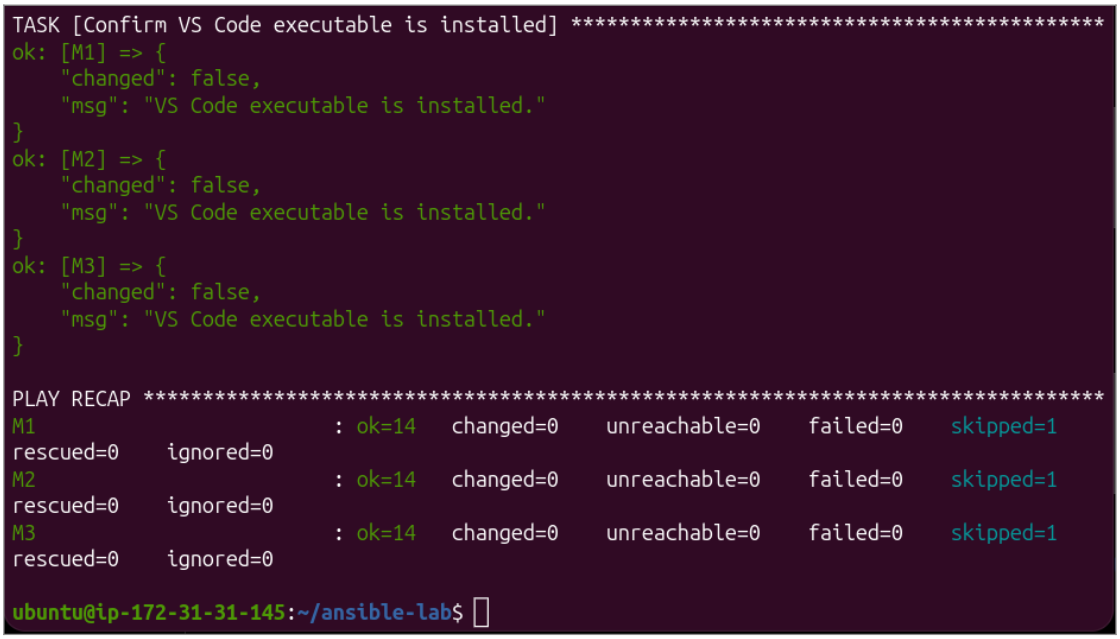


Figure 36: *Playbook output ending with the PLAY RECAP for M1, M2 and M3.*

11. Verifying the Installations from the Central Node

As a final check, I used ad hoc Ansible commands from the central node to confirm directly that Docker and VS Code exist on all managed nodes, without logging in to each server.

11.1 Checking the Docker Version

Command used

```
ansible managed -i inventory.ini -b -m ansible.builtin.command -a "docker --version"
```

Command explanation

- `ansible managed` targets every host of the `managed` group.
- `-i inventory.ini` selects the inventory file.
- `-b` (become) runs the command with administrator rights.
- `-m ansible.builtin.command` uses the `command` module, which runs a command on the remote node.
- `-a "docker --version"` is the argument: the command to run on each node.

Result

Each node answers with `rc=0`, which means the command succeeded, and prints `Docker version 29.9.0, build f415da8`. Ansible labels the result `CHANGED` because the `command` module reports a change by default; this does not mean anything was modified. All three managed nodes run the same Docker version.

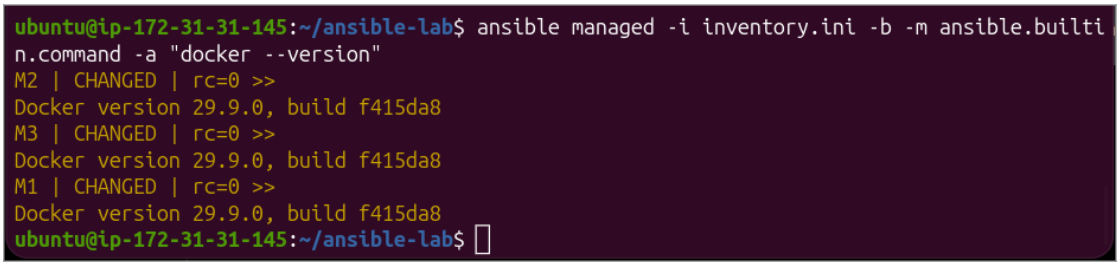


Figure 37: *Checking the Docker version on M1, M2 and M3 with an ad hoc Ansible command.*

11.2 Checking the VS Code Executable

Command used

```
ansible managed -i inventory.ini -m ansible.builtin.command \
  -a "ls -l /opt/VSCode-linux-x64/code"
```

Command explanation

This uses the same pattern as before, but the command run on each node is `ls -l /opt/VSCode-linux-x64/code`, which lists the VS Code executable in long format.

Result

Every node returns `rc=0` and the line `-rwxr-xr-x 1 ubuntu ubuntu 223685880 Oct 2 21:02 /opt/VSCode-linux-x64/code`. The file exists at the expected path, it is executable (`x` permissions) and has a size of 223685880 bytes, which confirms that VS Code is installed on M1, M2 and M3.

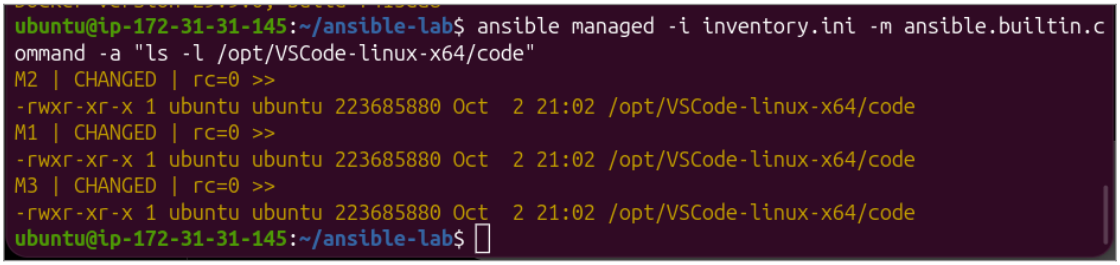


Figure 38: *Checking the VS Code executable on the managed nodes.*

12. Ansible Version and Final Instance Status

12.1 Ansible Version on the Central Node

To document the exact tool version used in the lab, I checked the Ansible version on the central node.

Command used

```
ansible --version
```

Command explanation

`ansible --version` prints the installed Ansible version together with its configuration paths and the Python and library versions it uses.

Result

The output shows `ansible [core 2.20.1]` installed at `/usr/bin/ansible`, running on Python 3.14.4 with Jinja 3.1.6 and PyYAML 6.0.3. It reports `config file = None`, so no custom configuration file is used and the inventory is passed with `-i` in every command.

```
ubuntu@ip-172-31-31-145:~/ansible-lab$ ansible --version
ansible [core 2.20.1]
  config file = None
  configured module search path = ['/home/ubuntu/.ansible/plugins/modules', '/usr/share/ansi
ble/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/ubuntu/.ansible/collections:/usr/share/ansible/collect
ions
  executable location = /usr/bin/ansible
  python version = 3.14.4 (main, Apr  8 2026, 04:02:31) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
ubuntu@ip-172-31-31-145:~/ansible-lab$
```

Figure 39: *Ansible version information on the central node.*

12.2 Final State of the EC2 Instances

At the end of the lab I returned to the EC2 console to check that all four servers were still healthy.

Result

EC2-Central, M1, M2 and M3 are all **Running** and show **3/3 checks passed**. M1, which was still initialising in Figure 7, now passes all of its status checks as well.

EC2-Central	i-0aa2fb7ddc1deaca8	Running	t3.micro	3/3 checks passec	eu-north-1a	ec2-13-60-237-46.eu-n.
M1	i-0eb6f5b591f7a03ad	Running	t3.micro	3/3 checks passec	eu-north-1a	ec2-51-20-136-251.eu-.
M2	i-0148a111312db5036	Running	t3.micro	3/3 checks passec	eu-north-1a	ec2-13-61-151-25.eu-n.
M3	i-0ab1163753f2e5f9c	Running	t3.micro	3/3 checks passec	eu-north-1a	ec2-13-60-2-135.eu-no.

Figure 40: *Final EC2 instance list showing all four instances running with 3/3 checks passed.*

13. Conclusion

In this lab I built a small Ansible environment on AWS. I launched a central node and three managed nodes, restricted SSH access with security groups, and installed Ansible on the central node. I wrote an inventory file, proved connectivity with the `ping` module, and installed Docker and VS Code on the central node by hand. Then I wrote the playbook `install_tools.yml`, which installed Docker and VS Code on M1, M2 and M3 in one

run. The PLAY RECAP showed no failed or unreachable hosts, and ad hoc commands confirmed Docker 29.9.0 and the VS Code executable on every managed node.

The lab showed me how configuration management removes repetitive manual work: the same playbook can be run again on any number of servers and always leads to the same result.

Component	Where	Evidence
Ansible core 2.20.1	EC2-Central	`ansible --version` (Figure 39)
Docker Engine	EC2-Central	`hello-world` container ran successfully (Figure 26)
VS Code 1.141.0	EC2-Central	`code --version` (Figure 28)
Docker 29.9.0	M1, M2, M3	Playbook output and ad hoc command (Figures 34 and 37)
VS Code executable	M1, M2, M3	`/opt/VSCode-linux-x64/code` found on each node (Figure 38)